

Client Interface (CIF) Full Microarchitecture Specification

Contents

1 Overview

The Client Interface (CIF) accepts AXI read and write transactions, validates them, divides each accepted transaction into up to 64 MC request packets (one per 64-B AXI data beat), and returns AXI responses in order for each AXID. Read and write traffic use independent instances of the same transaction-tracking ROB architecture.

1.1 Global assumptions

- AXID is 6 bits (64 AXI IDs); each ROB instance has 64 per-AXID Slot FIFOs, each of depth four.
- One MC request packet carries one 512-bit / 64-B AXI data beat. One accepted AXI transaction produces up to 64 MC packets — a beat *is* an MC packet.
- Every AXI data beat transfers a full 64-B unit (**AxSIZE** = 6) at a 64-B-aligned address. Like the other AXI-protocol constraints, this is a design-time assumption verified by assertion, not a runtime check (see the Request Validator); it is what makes one AXI beat correspond to exactly one 64-B MC packet and **exp_pkts** = **AxLEN** + 1.
- An AXI transaction must not cross 4 KB, so it produces at most **MAX_PKTS** = 4 KB / 64 B = 64 MC packets. With runtime-configured **row_size** of at least 1 KB, row crossings are at most three and affect each packet’s address, not the packet count.
- MC Core echoes the packet’s **GLOBAL_TAG** on completion, one response per accepted 64-B request packet (§??). Completion **status** encoding is frozen: 0/1/2 = OK/SLVERR/DECERR (§??).

2 Parameters

Parameter	Value
AXID_W	6
ROB_DEPTH	256 transactions per ROB instance
ROB_INDEX_W	8
DATA_W	512: one 64-B AXI data beat per MC packet
PACKET_NUMBER_W	6
GLOBAL_TAG_W	14: {ROB_INDEX[7:0], PACKET_NUMBER[5:0]}
COMPLETION_BITMAP_W	64
MAX_PKTS	64
Per-AXID Slot FIFO	64 FIFOs × depth 4, per ROB instance
Read Data Buffer watermark	128 entries, owned independently by that buffer

3 Transaction identity and packet formation

At AXI transaction acceptance, the applicable ROB Free List allocates **ROB_INDEX[7:0]**. The index is pushed into the accepting AXID’s Slot FIFO only after the Burst Splitter has supplied **exp_pkts**, the number of 64-B MC packets for the transaction (1..64). The ROB initializes **exp_pkts** and clears its 64-bit completion bitmap before the entry becomes eligible for retirement.

At packet emission, the Burst Splitter assigns **PACKET_NUMBER** (0..**exp_pkts**-1) and Packet Formation attaches **GLOBAL_TAG** = {**ROB_INDEX[7:0]**, **PACKET_NUMBER[5:0]**} (14 bits). This is the

complete MC completion identity.

4 Write path

4.1 Request path

The AW Metadata FIFO, W Data FIFO, Request Validator, Error Handler, Burst Splitter, Packet Formation Unit, and Write Packet FIFO retain their existing roles. The accepting write ROB allocates the transaction index. The splitter generates up to 64 MC packets, one per 64-B AXI data beat, and the Write Packet FIFO forwards them to MC Core.

4.2 Write ROB and response

The write ROB has its own 256-entry Free List, Metadata RAM, and 64 Slot FIFOs. Its Metadata RAM is directly indexed by `ROB_INDEX` and stores `exp_pkts[6:0]`, `completion_bitmap[63:0]`, aggregate `resp_status[1:0]`, and the Issue-#11 footprint `wr_line_base[25:0]/wr_line_count[6:0]` (§??). MC completions set the bit selected by `PACKET_NUMBER` and update aggregate status. The `status` encoding (0/1/2 = OK/SLVERR/DECERR, codes 3..15 reserved → DECERR) and the worst-status precedence (`DECERR > SLVERR > OKAY`) are frozen in §??.

When the head transaction of an AXID Slot FIFO has `completion_bitmap` bits [`exp_pkts-1:0`] all set, retirement selects it in AXID order. It holds BRESP valid until the downstream AXI response path accepts it. Only that acceptance pops the Slot FIFO and returns `ROB_INDEX` to the write Free List. Write traffic has no Read Data Buffer.

5 Read path

5.1 Request path

The AR Metadata FIFO, Request Validator, Error Handler, Burst Splitter, Packet Formation Unit, and Read Packet FIFO retain their existing roles. The read ROB allocates `ROB_INDEX` at acceptance, subject to the same-line read/write hazard check at the AR Metadata FIFO head (§??). The splitter emits up to 64 MC packets, each carrying a distinct `PACKET_NUMBER` (0..`exp_pkts-1`).

5.2 Read ROB, Data Buffer, and serializer

The independent read ROB has the same generic structures as the write ROB. Its Metadata RAM additionally stores `rd_dbuf_base`, the base of a single contiguously-allocated RD_SRAM range of `exp_pkts` 64-B lines, set at allocation. On completion, packet `PACKET_NUMBER`'s returned 64-B data unit is written to `rd_dbuf_base + PACKET_NUMBER` and `completion_bitmap[PACKET_NUMBER]` is set.

Once a Slot FIFO head is complete, a Retiring Register holds its `ROB_INDEX` while the Read Serializer emits `rd_dbuf_base + 0 .. rd_dbuf_base + exp_pkts-1` in packet order, one AXI beat per 64-B MC packet. `RLAST` is asserted on the final beat (`PACKET_NUMBER == exp_pkts - 1`); only after that beat is accepted and the `rd_dbuf_base` range released may the Slot FIFO be popped and `ROB_INDEX` returned to the read Free List. RD_SRAM depth (hence `RD_SRAM_ADDR.W`), its handshakes and latency remain **TBD**; its 128-entry watermark is its own capacity control, not ROB occupancy control.

5.3 Same-line read/write ordering

The MC core may reorder a read ahead of an older write to the same 64 B line. CIF prevents the resulting stale read by holding a conflicting read at admission until the overlapping write has retired. The check is request-wise and uses only the raw 64 B line address; no hashed `daddr` and no CAM are introduced.

Write footprint. Each outstanding write is tracked in its `ROB_WR` entry as one contiguous 64 B line interval $\{\text{wr_line_base}[25:0], \text{wr_line_count}[6:0]\}$, derived from $\{\text{AWADDR}, \text{AWLEN}, \text{AWSIZE}, \text{AWBURST}\}$:

- `nbytes = (AWLEN + 1) << AWSIZE`
- `INCR: start = AWADDR; last = AWADDR + nbytes - 1`
- `WRAP: start = AWADDR & ~(nbytes - 1); last = start + nbytes - 1`
- `FIXED: start = AWADDR; last = AWADDR + (1 << AWSIZE) - 1`
- `wr_line_base = start[31:6]`
- `wr_line_count = last[31:6] - start[31:6] + 1, range [1,64]`

Rounding outward to line granularity is mandatory: burst alignment and `AWSIZE` are not checked at runtime. `wr_line_count` ≤ 64 follows from the 4 KB transaction limit (§??). The footprint is written at allocation (§??), is never modified by completion, and is released when the entry is freed (§??).

Read admission check. At the head of the AR Metadata FIFO, on the Request Validator valid path and *before* `alloc_req`, the incoming read’s footprint is computed from $\{\text{ARADDR}, \text{ARLEN}, \text{ARSIZE}, \text{ARBURST}\}$ by the identical calculation and tested for interval overlap against every occupied `ROB_WR` entry. Writing R for the incoming read footprint and W for an occupied write footprint, the read conflicts iff

$$(R_{\text{base}} < W_{\text{base}} + W_{\text{count}}) \wedge (W_{\text{base}} < R_{\text{base}} + R_{\text{count}}).$$

The read footprint is transient: used for the check only, stored nowhere. No footprint or hazard state is added to `ROB_RD`, and a completed read is never re-checked.

Hold and release. On any overlap the AR Metadata FIFO head is not popped and no read `ROB_INDEX` is allocated; the hold uses the same mechanism as the existing per-AXID `STALL` (outputs held stable, FIFO not advanced). The check is re-evaluated whenever a write `ROB_INDEX` is freed (`free_req`, §??); the freed entry’s footprint stops being occupied in the same cycle its Free List bit is set. The read admits as soon as it overlaps no occupied write footprint. Liveness holds because writes drain independently of the held read. Holding the FIFO head also stalls non-conflicting reads behind it on that port; this is accepted, as same-line overlap is rare.

MC contract dependency. Release keys on the blocking write’s `ROB_WR` entry being freed, which occurs when its `BRESP` is accepted after every `WR_ACK` for that write has been received. This is correct only if a write reported complete is visible to a subsequent same-line read; that property is stated as an explicit interface requirement on MC Core (§??, item 2).

6 CIF–MC Interface Contract

This section is the authoritative statement of the $\text{CIF} \rightarrow \text{MC}$ and $\text{MC} \rightarrow \text{CIF}$ interface. Where an earlier section gives a placeholder signal name or leaves an interface detail open, this section governs. It applies the frozen decisions of §??, §??, §??, and §?? without changing them.

6.1 Channels

Three clock-domain-crossing FIFOs cross the CIF–MC boundary:

FIFO	Direction	Payload
req_afifo	CIF → MC	{req_type[0], GLOBAL_TAG[13:0], daddr[DADDR.W-1:0]}
wdata_afifo	CIF → MC	{wdata[511:0], wmask[63:0]} — write packets only
resp_afifo	MC → CIF	{resp_type[0], GLOBAL_TAG[13:0], status[3:0], rdata[511:0]}

`req_afifo` is one ordered stream carrying read and write descriptors interleaved in CIF packet-emission order; that order is program order. `wdata_afifo` carries one entry per write descriptor, in the same order the write descriptors enter `req_afifo` (positional binding); a write packet is emitted only when both request- and write-data credit are available. `resp_afifo` is one merged stream; `rdata` is valid only when `resp_type` = `RD_DATA`.

6.2 Request packet

Field	Width	Meaning
req_type	1	0 = READ, 1 = WRITE. Invalid AXI requests never cross (see §?? error transactions)
GLOBAL_TAG	14	{ROB_INDEX[7:0], PACKET_NUMBER[5:0]} (§??); opaque to MC, echoed verbatim on the response
daddr	DADDR.W	DRAM coordinates {rank, ch, bg, bank, row, col}, MSB→LSB, produced by the CIF-side address map/hash. <i>Not</i> a byte address; sub-block / offset bits excluded. DADDR.W = RANK_BITS + CH_BITS + 3 + 2 + ROW_BITS + COL_BITS (BG_BITS = 3, BANK_BITS = 2 frozen); baseline 30

One request packet is one 64-B AXI data beat; up to `MAX_PKTS` = 64 per transaction. `PACKET_NUMBER` is the Burst Splitter `pkt_idx`, 0..`exp_pkts`-1. There is no `beat_cnt`, `size`, or `len` field. For WRAP bursts the CIF-side WRAP Beat Address Table produces the per-packet byte address that is then mapped to `daddr`.

6.3 Write-data packet

Field	Width	Meaning
wdata	512	one 64-B write beat
wmask	64	byte-enable, bit <i>i</i> ↔ byte <i>i</i> , verbatim from AXIWSTRB. <code>wmask</code> not all-ones ⇒ MC read-modify-write

6.4 Response packet

Field	Width	Meaning
resp_type	1	0 = RD_DATA (route to read ROB), 1 = WR_ACK (route to write ROB)
GLOBAL_TAG	14	echoed verbatim; CIF decodes ROB_INDEX = [13:6], PACKET_NUMBER = [5:0]
status	4	0 = OK, 1 = SLVERR, 2 = DECERR. Codes 3..15 reserved; CIF treats a reserved value as DECERR
rdata	512	RD_DATA only; undefined for WR_ACK

One MC completion sets exactly one `completion_bitmap` bit, one per 64-B packet. A packet completing is not RLAST: RLAST additionally requires `PACKET_NUMBER == exp_pkts - 1` and that beat to have been serialized and accepted downstream.

6.5 Flow control

Request side (`req_afifo`, `wdata_afifo`). Credit-based push, one independent credit pool per FIFO. CIF holds `N_REQ_CREDITS` (equal to the FIFO depth), drives a push while credit > 0, and decrements per push. MC returns one `credit_return` pulse per consumed entry (registered, 2-flop synchronized MC → CIF). No combinational ready crosses the clock-domain boundary; there is no `PKT_READY`.

Response side (`resp_afifo`). Valid-only push; no ready from CIF. MC self-throttles: every read reserves a `resp_afifo` slot at MC issue time and is not issued unless a slot is free. CIF accepts every response the cycle it is presented (one per cycle, merged stream, dedicated Metadata-RAM completion port) and returns a response credit to MC only after the response is fully processed — the `completion_bitmap` bit set, and for RD_DATA the `rdata` written to `rd_dbuf_base + PACKET_NUMBER`.

6.6 Transaction identity

The 14-bit `GLOBAL_TAG` is the sole end-to-end identifier. MC stores it verbatim and never interprets any sub-field. No AXI ID crosses the interface; CIF recovers the AXID from per-AXID Slot-FIFO context at retirement (§??).

6.7 Status to AXI response

status	AXI RESP[1:0]
0	OKAY (2'b00)
1	SLVERR (2'b10)
2	DECERR (2'b11)
3..15	reserved → DECERR

`status` is per-packet. **Read:** each packet's mapped `RESP` drives `RRESP` for the AXI R beat it serializes into — no aggregation. **Write:** `resp_status` is the worst-of over the transaction's `exp_pkts` `WR_ACKs`, precedence `DECERR > SLVERR > OKAY` (numeric max of the `status` codes); the single AXI `BRESP` is `map(resp_status)`. `EXOKAY` is unused.

6.8 Error transactions

Invalid AXI requests (CIF address-range-check failures) are completed entirely inside CIF and *do not cross the interface*. The Error Handler drives `alloc_req`; at Metadata-RAM initialization it force-fills `completion_bitmap[exp_pkts-1:0]` and sets `resp_status = SLVERR`, so the entry is immediately retirement-eligible and retires in Slot-FIFO order. A read error emits `exp_pkts` R beats with `RRESP = SLVERR` and undefined data, `rob_last` on the final beat. MC-detected faults on *valid* transactions are carried in the `status` field of the owning `RD_DATA/WR_ACK`, not as a distinct response type.

6.9 Requirements on MC Core

The two items below are **requirements this interface places on MC Core**. They are stated as interface dependencies; the current MC-core documentation is *not* asserted to implement or to have verified them.

1. **Response granularity.** MC shall generate *exactly one* response packet for every accepted 64-B request packet, echoing that packet's `GLOBAL_TAG` unmodified. No burst-level or transaction-level response grouping. There is no `last` field on the interface — CIF derives `RLAST` from `PACKET_NUMBER == exp_pkts - 1`. Response packets for one transaction may arrive in any order; CIF's `completion_bitmap` and `rd_dbuf_base + PACKET_NUMBER` placement tolerate that.
2. **Write-completion safety (Issue #11 dependency).** MC shall issue `WR_ACK` for a write packet only when that packet is complete *and* a subsequent read to the same 64-B address is guaranteed to observe the write. CIF releases a held same-line read (§??) when the blocking write's `ROB_INDEX` is freed — which occurs once every `WR_ACK` for that write has been accepted and its `BRESP` emitted. The correctness of Issue #11 therefore depends on this requirement; CIF adds no further MC-visibility guarantee of its own.

6.10 Ordering

`req_afifo` order is program order; MC derives its internal ordering token from ingress order. MC may schedule DRAM commands and return responses out of order. MC does not reorder the response stream for AXI purposes; CIF owns AXI R- and B-channel ordering per AXID (§??).

7 Ordering, capacity, and reset

Slot FIFO order is transaction acceptance order for each AXID and therefore enforces AXI response ordering. Metadata RAM capacity is transaction capacity only. A full Slot FIFO blocks allocation for that AXID; an empty Free List blocks allocation globally. The latter means all 256 transaction slots are allocated, not merely that one Slot FIFO is full. Read request issuance is controlled separately by pooled Read Data Buffer availability.

Reset is active-low, asserted asynchronously, and deasserted synchronously; §?? gives the full initialization sequence. It clears both ROB Free Lists, Metadata RAM state, Slot FIFOs, and Retiring Registers. The Read Data Buffer is reset and drained according to its own specification. A soft reset is entered only after outstanding responses have been accepted and both ROB instances have no allocated Slot FIFO entries.

8 Error handling

Invalid requests (CIF address-range-check failures) are completed entirely inside CIF and never cross the CIF–MC interface (§??, error transactions). The Error Handler allocates the ROB entry, force-fills `completion_bitmap` and sets `resp_status = SLVERR` at initialization, and the entry retires in Slot-FIFO order. The `status` → AXI RESP mapping is frozen in §?? (0/1/2 = OK/SLVERR/DECERR); only the meaning of MC error codes 3..15 remains an MC-owned detail.

9 Top-level AXI4 interface

CIF presents a standard AXI4 slave interface to its client(s). The table below cross-references the channel signals against the widths the current architecture documents establish (`ADDR_W=32`, `AXID_W=6`, `LEN_W=8`, from the Request Validator’s Design Parameters). Widths marked AXI4-fixed follow the AXI4 protocol specification itself and are not CIF-specific parameters. The write and read data paths are 512 bits: one 64-B AXI data beat per MC request packet (§??, §??).

Channel	Signal	Width	Notes
AW	AWVALID/AWREADY	1/1	Write address handshake
	AWADDR	32	<code>ADDR_W</code>
	AWID	6	<code>AXID_W</code>
	AWLEN	8	<code>LEN_W</code> ; raw AXI burst length. Given the 64-B-beat contract, <code>exp_pkts = AWLEN + 1</code> (§??)
	AWSIZE/AWBURST	3/2	AXI4-fixed; also feed the write 64 B line footprint calc (§??)
W	WVALID/WREADY	1/1	Write data handshake
	WDATA	512	One 64-B AXI data beat per MC request packet (§??)
	WSTRB	64	AXI4-fixed (one bit per <code>WDATA</code> byte); carried verbatim as the <code>wmask</code> field on <code>wdata_afifo</code> (§??)
	WLAST	1	AXI4-fixed
B	BVALID/BREADY	1/1	Write response handshake
	BID	6	<code>AXID_W</code> ; from the retiring transaction’s Slot FIFO context, not stored in the Metadata RAM entry
	BRESP	2	AXI4-fixed; driven from the write ROB’s aggregate <code>resp_status</code>
AR	ARVALID/ARREADY	1/1	Read address handshake
	ARADDR	32	<code>ADDR_W</code>
	ARID	6	<code>AXID_W</code>
	ARLEN	8	<code>LEN_W</code> ; raw AXI burst length. Given the 64-B-beat contract, <code>exp_pkts = ARLEN + 1</code> (§??)
	ARSIZE/ARBURST	3/2	AXI4-fixed; also feed the read footprint used by the same-line hazard check (§??)
R	RVALID/RREADY	1/1	Read data handshake
	RDATA	512	One 64-B AXI data beat per MC request packet (§??)
	RID	6	<code>AXID_W</code> ; from the retiring transaction’s Slot FIFO context

RRESP	2	AXI4-fixed; per-beat value from the read ROB's serialized status
RLAST	1	Asserted only on the final AXI beat of the transaction (<code>PACKET_NUMBER == exp_pkts - 1</code>), not on every packet

Table 1: Top-level AXI4 slave interface, cross-referenced to CIF-internal handling

10 Interface Signal Table

This table records the consolidated interfaces after the read/write ROB refactor. Names describe the frozen connectivity where prior documents did not fix a signal name.

10.1 AXI request acceptance to the selected ROB

Signal	Width	Path	Description
<code>alloc_req</code>	1	Accept logic → RD/WR ROB	Allocate one transaction slot on accepted AR/AW request.
<code>AXID</code>	6	Accept logic → RD/WR ROB	Selects the per-AXID Slot FIFO.
<code>ROB_INDEX</code>	8	RD/WR ROB → Splitter	Free-List result retained for the transaction.
<code>exp_pkts</code>	7	Splitter → RD/WR ROB	Number of 64-B MC packets for this transaction, 1–64; initializes the ROB's <code>exp_pkts</code> .
<code>wr_line_base</code>	26	Accept logic → WR ROB	Start 64 B line of the write footprint; written at allocation (§??).
<code>wr_line_count</code>	7	Accept logic → WR ROB	Contiguous 64 B line span of the write, 1–64 (§??).
<code>HAZ_HOLD</code>	1	Same-line hazard gate → Request Validator	Read path: head read overlaps an occupied ROB_WR footprint; withholds <code>alloc_req</code> (§??).
<code>free_req</code>	—	WR ROB → same-line hazard gate	Existing write-retirement pulse; also re-arms the held-read recheck (§??).

10.2 Burst Splitter to Packet Formation

Signal	Width	Path	Description
<code>emit_valid/emit_ready</code>	1	Splitter ↔ Packet Formation	Handshake for the current 64-B MC packet.
<code>emit_addr</code>	32	Splitter → Packet Formation	64-B-granular start address of this packet's data beat.
<code>PACKET_NUMBER</code>	6	Splitter → Packet Formation	Current <code>pkt_idx</code> (<code>0..exp_pkts-1</code>), assigned at packet emission.

GLOBAL_TAG	14	Packet Formation → req.afifo	{ROB_INDEX[7:0], PACKET_NUMBER[5:0]}.
------------	----	------------------------------	---------------------------------------

Packet Formation applies the CIF-side address map/hash to `emit_addr`, producing the packet's `daddr` field (§??); the raw byte address is not carried to MC.

10.3 MC Core completion interface

Response transport is the valid-only `resp.afifo` (§??); CIF always accepts, MC self-throttles via a per-read reserved slot. MC Core supplies exactly one response per accepted 64-B request packet (§??).

Signal	Width	Path	Description
<code>resp_type</code>	1	MC Core → RD/WR ROB	0 = RD_DATA (route to read ROB), 1 = WR_ACK (route to write ROB).
GLOBAL_TAG	14	MC Core → RD/WR ROB	Echoed request tag; [13:6] = ROB_INDEX, [5:0] = PACKET_NUMBER.
<code>status</code>	4	MC Core → RD/WR ROB	0 = OK, 1 = SLVERR, 2 = DECERR; codes 3..15 reserved → DECERR (§??).
<code>rdata</code>	512	MC Core → Read ROB	Read data payload, one 64-B unit; valid only when <code>resp_type</code> = RD_DATA (§??).

10.4 Retirement interfaces

Signal	Width	Path	Description
<code>merge_ready</code>	1	Merge Logic → Read ROB	Accepts an emitted AXI read beat (one per 64-B MC packet).
<code>rob_last</code>	1	Read ROB → Merge Logic	Asserted with the final serialized read beat (<code>PACKET_NUMBER == exp_pkts - 1</code>).
<code>bresp_ready</code>	1	B response path → Write ROB	Accepts the ordered BRESP and permits write-slot release.
<code>rd_dbuf alloc/read/free</code>	TBD	Read ROB ↔ Read Data Buffer	Handshake and release timing are TBD ; a contiguous <code>exp_pkts</code> -line range is allocated at ROB init and released as one unit.

11 Arbitration and backpressure

The request validators and Burst Splitter retain the existing request arbitration policy. The selected ROB Free List and selected AXID Slot FIFO provide transaction-allocation backpressure. A full Slot FIFO affects only its AXID; it does not imply global Free List exhaustion. Conversely, an empty Free List means all 256 transaction entries are allocated.

Read packet issuance is additionally controlled by pooled Read Data Buffer availability and its independent 128-entry watermark. Metadata RAM capacity does not control packet issue.

12 Reset and initialization sequence

Reset is active-low, asserted asynchronously, and deasserted synchronously to the CIF clock domain. While asserted, it forces every structure below to its initialized state; normal operation begins only after synchronous deassertion.

On reset, each ROB instance independently initializes its 256-bit Free List to free, clears Metadata RAM state, resets all 64 Slot FIFOs, and clears its Retiring Register. The write `wr_line_base` and `wr_line_count` footprint fields are part of Metadata RAM state and are cleared with it; the read-admission same-line hazard gate is combinational and holds no reset state (§??). The Burst Splitter clears its per-packet address-layout state, `exp_pkts`, `pkt_idx`, and retained `ROB_INDEX`. The CIF–MC async FIFOs (§??) reset to empty; each credit counter resets to its FIFO depth. No transaction crosses either seam until both clock domains are out of reset. The address-map / hash CSRs are CIF-side and are loaded and locked before `init_done`.

A soft reset first quiesces request acceptance and waits for packet FIFOs, Burst Splitter activity, both ROB Slot-FIFO sets, both Retiring Registers, and all externally visible AXI responses to drain. A read held at the AR Metadata FIFO head by the same-line hazard check (§??) is upstream of ROB allocation; quiescing request acceptance stops feeding it and its blocking writes drain through normal retirement, so no additional quiescence condition is required. The Read Data Buffer has a separate drain/initialization contract; its exact interface and timing are **TBD**. Reset must not release a read `ROB_INDEX` until the final AXI read response and its Data Buffer release have completed.

13 Error propagation

Error packets are split and tagged in the same manner as ordinary requests. Every completion sets its packet-number bit in the selected ROB. The write ROB aggregates packet status before its one BRESP; the read ROB serializes status with its data. The `status` encoding (0/1/2 = OK/SLVERR/DECERR, codes 3..15 reserved → DECERR) and the write worst-status precedence (DECERR > SLVERR > OKAY) are frozen in §??.

A Burst Splitter

A.1 Purpose and bounds

The Burst Splitter converts one accepted AXI transaction into up to 64 MC request packets, one per 64-B AXI data beat. It is shared by the read and write request paths. Each packet carries one 512-bit / 64-B data unit; there is no `beat_cnt` field in the packet.

The configured `row_size` is a run-time parameter and is at least 1 KB. AXI4 prohibits a burst from crossing a 4 KB boundary, so a transaction spans at most 4 KB, i.e. at most

$$\text{MAX_PKTS} = 4 \text{ KB} / 64 \text{ B} = 64$$

64-B MC packets. A transaction can still cross at most three 1 KB row boundaries; row crossings affect each packet’s address, not the packet count.

A.2 Interfaces and state

At transaction acceptance, the selected read or write ROB allocates `ROB_INDEX[7:0]`. The splitter accepts that index with the AXI metadata and retains it for the whole transaction. Stage 1 determines `exp_pkts`, the number of 64-B MC packets, and presents it to the corresponding ROB; this is the sole source of the ROB's `exp_pkts`. Because every AXI data beat is one 64-B unit (§??), `exp_pkts = AxLEN + 1` for every transaction — INCR, WRAP, and FIXED alike.

State	Width/depth	Meaning
<code>ROB_INDEX</code>	8 bits	Transaction index retained while packets issue
<code>exp_pkts</code>	7 bits	Number of 64-B MC packets for the transaction, 1..64
<code>pkt_idx</code>	6 bits	<code>PACKET_NUMBER</code> of the packet currently being emitted, 0.. <code>exp_pkts</code> -1

The internal structure Stage 1 uses to record the per-packet address layout (a per-beat address generator, and how row boundaries are tracked within it) is not fixed by this document.

A.3 Stage 1 — packet-layout construction

Stage 1 receives address, burst length, size, burst type, AXID, packet type, and `ROB_INDEX`. It computes `exp_pkts` — the number of 64-B MC packets the transaction requires, equal to `AxLEN + 1` given the 64-B-beat contract (§??) — and the per-packet address layout. For an INCR transaction packet `N` addresses byte `base + 64 N`. The AXI 4 KB constraint bounds `exp_pkts` at 64.

For a read allocation, Stage 1 provides `exp_pkts` to the read ROB before its `ROB_INDEX` becomes visible at the relevant per-AXID Slot FIFO. The write path follows the identical ordering.

A.4 Stage 2 — sequential emission

Stage 2 emits the transaction's packets in increasing `pkt_idx` order. For each accepted packet it forms

$$\text{GLOBAL_TAG}[13:0] = \{\text{ROB_INDEX}[7:0], \text{PACKET_NUMBER}[5:0]\}, \quad \text{PACKET_NUMBER} = \text{pkt_idx}.$$

Packet Formation receives the packet's 64-B-granular `addr`, packet type, and `GLOBAL_TAG`; it serializes one MC request packet, carrying one 512-bit / 64-B data unit (write) or none (read). No per-packet Read Data Buffer handle is recorded — the read ROB's contiguous `rd_dbuf_base` range was allocated at ROB initialization and packet `pkt_idx`'s data lands at `rd_dbuf_base + pkt_idx`.

Stage 2 advances `pkt_idx` only on downstream packet acceptance. It stalls Stage 1 for the duration of a multi-packet transaction, preserving the packet layout and its associated `ROB_INDEX`. After the final accepted packet it returns to idle and permits the next Stage 1 transaction.

A.5 WRAP addressing

For an INCR transaction, packet `N`'s address is simply `base + 64 N`. For a WRAP transaction the beat addresses are not contiguous beyond the wrap boundary, so each packet's address is taken from the WRAP Beat Address Table — one entry per beat, i.e. one per 64-B MC packet, consumed in `pkt_idx` order. `exp_pkts` and the per-packet `PACKET_NUMBER` assignment are identical for INCR and WRAP; only the address source differs.

A.6 Backpressure and error packets

- A full selected ROB Slot FIFO, an empty selected ROB Free List, or unavailable read Data Buffer capacity prevents acceptance upstream as appropriate to that path.
- Packet-FIFO backpressure holds the current packet and its `pkt_idx`; it does not alter `exp_pkts`.
- Error read and error write transactions use the same per-packet `GLOBAL_TAG` mechanism. Their packet type is carried in every generated packet.

A.7 Timing notes

- `exp_pkts` is the number of 64-B MC packets generated by this one transaction; it equals the transaction's 64-B AXI data-beat count.
- `PACKET_NUMBER (= pkt_idx)` is assigned at packet emission, not allocation.

B Request Validator

B.1 Overview

The Request Validator is a purely combinational block instantiated once each on the read and write paths. It inspects the head entry of the AR or AW Metadata FIFO, performs a runtime address range check (and, on the read path, drives the same-line read/write hazard gate of §??), and steers the transaction to one of two downstream paths:

- **Valid path:** Transaction forwarded to ROB allocation with `REQ_VALID` asserted.
- **Error path:** Transaction forwarded to the Error Handler with `ERR_VALID` asserted and `AXID` preserved.

The Request Validator contains a Configuration Register that stores the valid physical address range used for runtime address validation. Apart from this internal configuration state, the Request Validator contains no per-transaction storage or counters; the read-path hazard gate is likewise combinational (§??). All protocol-level checks (burst type, WRAP legality, alignment, burst length, and the 64-B transfer size `AxSIZE = 6` with 64-B-aligned addressing that the MC packet contract requires) are enforced as design-time assertions in the verification environment only. Compliant AXI4 masters are assumed at runtime.

B.2 Design Parameters

Parameter	Value	Description
<code>ADDR_W</code>	32	Address width in bits
<code>AXID_W</code>	6	AXI ID width
<code>LEN_W</code>	8	Burst length field width (<code>ARLEN</code> / <code>AWLEN</code>)

Table 6: Request Validator parameters

B.3 Internal Blocks

Configuration Register Stores the valid physical address range used during request validation. Provides the configured address range to the Address Range Check block. Represents the only persistent state within the Request Validator.

B.4 Address Range Check

- **Type:** Purely combinational.
- **Input:** ADDR[31:0] from the AR/AW Metadata FIFO head.
- **Function:** Checks whether ADDR falls within the valid physical address range stored in the Configuration Register.
- **Output:** addr_ok (1-bit) — asserted when the address is valid, deasserted on an address error.
- **Scope:** Address range check only. No burst type, no alignment, no length checks at runtime.

B.5 Output Mux

- **Type:** Purely combinational.
- **Function:** Routes the transaction to the correct downstream block based on addr_ok.
- **Valid path** (addr_ok == 1): asserts REQ_VALID to ROB allocation; forwards AXID[5:0].
- **Error path** (addr_ok == 0): asserts ERR_VALID to the Error Handler; forwards AXID[5:0].
- Only one output path is active per transaction. Both paths are never asserted simultaneously.

B.6 Interface Signals

B.7 Inputs

Signal	Width	Source	Description
ADDR[31:0]	32	AR/AW Metadata FIFO	Transaction address
AXID[5:0]	6	AR/AW Metadata FIFO	AXI ID
ARLEN/AWLEN[7:0]	8	AR/AW Metadata FIFO	Burst length
ARSIZE/AWSIZE[2:0]	3	AR/AW Metadata FIFO	Transfer size; feeds the 64 B line footprint calc (
ARBURST/AWBURST[1:0]	2	AR/AW Metadata FIFO	Burst type; feeds the footprint calc (§??)
STALL	1	ROB Allocation	Per-AXID stall; holds output until deasserted
HAZ_HOLD	1	Same-line hazard gate	Read instance only. Head read overlaps an occu

Table 7: Request Validator input signals

B.8 Outputs

Signal	Width	Destination	Description
REQ_VALID	1	ROB allocation	Valid transaction request
AXID[5:0]	6	ROB allocation	Forwarded AXI ID
ERR_VALID	1	Error Handler	Address error notification
AXID[5:0]	6	Error Handler	AXI ID preserved for error-response handling

Table 8: Request Validator output signals

B.9 Functional Behaviour

1. Request Validator reads, without popping, the AR/AW Metadata FIFO head entry (`ADDR`, `AXID`, `ARLEN/AWLEN`, `ARSIZE/AWSIZE`, `ARBURST/AWBURST`; widths in the Inputs table).
2. Address Range Check compares the incoming address against the physical address range stored in the Configuration Register and evaluates `addr_ok` combinationaly.
3. If `addr_ok == 1` and `STALL` is deasserted (and, on the read path, `HAZ_HOLD` is deasserted): Output Mux asserts `REQ_VALID` to ROB allocation with `AXID`, and the FIFO head is popped in that cycle — the transaction has been accepted.
4. If `addr_ok == 0`: Output Mux asserts `ERR_VALID` to the Error Handler with `AXID`, and the FIFO head is popped when `ERR_VALID` is asserted. The Error Handler drives `alloc_req` and, at Metadata-RAM initialization, force-fills `completion_bitmap` and sets `resp_status = SLVERR`; the entry is immediately retirement-eligible. No packet is emitted to the CIF-MC interface (§??, error transactions).
5. If `STALL` is asserted (ROB Allocation unavailable for this `AXID`), or `HAZ_HOLD` is asserted on the read path (§??): the Request Validator holds its outputs stable and leaves the current head entry in place. The FIFO is not advanced until the transaction is accepted (step 3) or routed to the Error Handler (step 4).

B.10 Read/write same-line hazard gate

Read instance only. Before `alloc_req`, the head read's 64B line footprint — computed combinatorially from `{ARADDR, ARLEN, ARSIZE, ARBURST}` by the calculation in §?? — is tested for interval overlap against every occupied ROB_WR footprint (`wr_line_base`, `wr_line_count`, occupied bit). On any overlap the gate asserts `HAZ_HOLD`, which holds the Request Validator output and the AR Metadata FIFO exactly as `STALL` does. The overlap result is re-evaluated whenever a write ROB_INDEX is freed (`free_req`). The read admits when it overlaps no occupied write footprint. No read-side footprint is stored; see §?? for the full description and the MC-contract dependency.

B.11 Timing Notes

- Fully combinational. Output (`REQ_VALID` or `ERR_VALID`) is valid in the same cycle as the FIFO head entry is presented.
- `STALL` from ROB allocation is also combinational — the combined path (FIFO output → Address Range Check → Output Mux → ROB Allocation Stall Logic → `STALL` back) is a combinational loop that must be timed carefully. A register stage may be inserted between the Request Validator output and the ROB Allocation if timing closure requires it, at the cost of one cycle of allocation latency.
- The Address Range Check remains purely combinational. The Configuration Register provides the address range used during validation and is treated as stable during normal request processing.

B.12 Edge Cases

- **Error path stall:** An address error transaction still consumes a ROB allocation. If the `AXID` is at maximum outstanding (`STALL` asserted), the error path is also held. `ERR_VALID` is not

asserted until **STALL** deasserts.

- **Read held with allocation free:** On the read path a valid transaction may be held at the FIFO head by **HAZ_HOLD** (§??) even when **STALL** is deasserted. The write path is unaffected.
- **Protocol violations:** Illegal burst type, transfers that are not 64-B / **AxSIZE** = 6 or not 64-B-aligned, unaligned **WRAP**, and maximum burst length violations are not checked at runtime. They are enforced as design-time assertions in the verification environment. A non-compliant master will cause undefined behaviour.
- **Simultaneous valid and error:** Not possible — **addr_ok** is a single-bit result; exactly one output path is active per transaction.

C Transaction Allocation and ROB-Index Assignment

Transaction allocation is performed independently by the read and write ROB instances. The allocator’s result is an eight-bit **ROB_INDEX**.

C.1 Structures and parameters

Structure	Definition
Free List	256-bit free/occupied bitmap per ROB instance
Allocation result	ROB_INDEX [7:0] selected from a free bit
Ordering storage	64 per-AXID Slot FIFOs × depth 4 per instance
Completion identity	GLOBAL_TAG = { ROB_INDEX [7:0], PACKET_NUMBER [5:0]} (14 bits)
Write footprint	{ wr_line_base [25:0], wr_line_count [6:0]} per write entry (§??)

C.2 Allocation flow

1. Request acceptance selects either the read or write ROB and presents **alloc_req** and **AXID**. On the read path, **alloc_req** is asserted only after the same-line hazard check at the AR Metadata FIFO head passes (§??); on overlap the head is held and no index is reserved.
2. If that **AXID**’s Slot FIFO is full, allocation for that **AXID** stalls. If its Free List is empty, allocation stalls globally for that ROB instance.
3. The Free List selects a free **ROB_INDEX** and reserves it.
4. Burst Splitter Stage 1 produces **exp_pkts**, the number of 64-B MC packets for the transaction (1..64); the Metadata RAM entry is initialized with that count and a zero 64-bit completion bitmap. For the read ROB, **rd_dbuf_base** is set to the base of a contiguous **exp_pkts**-line **RD_SRAM** allocation. For the write ROB, the **wr_line_base** and **wr_line_count** footprint fields are written in the same step from {**AWADDR**, **AWLEN**, **AWSIZE**, **AWBURST**} (§??).
5. Only after initialization is the index pushed to the **AXID** Slot FIFO and made visible to retirement. The index then accompanies the transaction to the Burst Splitter.

C.3 Release flow

Completion is not release. The write ROB returns an index only after the ordered **BRESP** is accepted. The read ROB returns an index only after its final **AXI R** response is fully accepted and the corresponding contiguous **RD_SRAM** range has been released. Retirement performs the

Slot FIFO pop and Free List update; no external free-by-tag interface exists. Freeing a write index additionally re-arms the same-line hazard recheck for a held AR Metadata FIFO head (§??).

C.4 Removed legacy behavior

The Slot FIFOs enforce the depth-four per-AXID outstanding limit and preserve AXI response order. A full Slot FIFO does not imply that the 256 entry Free List is empty; global exhaustion occurs only when all transaction capacity is allocated.

D Write Completion Tracking and BRESP Retirement

Write completion tracking is implemented inside the write ROB instance described in §??.

D.1 Write ROB metadata

Each write Metadata RAM entry is directly addressed by `ROB_INDEX` and contains:

- `exp_pkts[6:0]`, the number of 64-B MC packets for the transaction (1..64, one per AXI data beat);
- `completion_bitmap[63:0]`, initially zero;
- `resp_status[1:0]`, initially OKAY; and
- `wr_line_base[25:0]` and `wr_line_count[6:0]`, the write's contiguous 64 B line footprint, set at allocation from `{AWADDR, AWLEN, AWSIZE, AWBURST}`, not touched by completion, and freed with the entry. It is read only by the read-admission same-line hazard check (§??).

For each MC completion, the write ROB decodes `GLOBAL_TAG` directly (`[13:6] = ROB_INDEX`, `[5:0] = PACKET_NUMBER`), sets `completion_bitmap[PACKET_NUMBER]`, and folds the packet status into `resp_status` as `resp_status ← max(resp_status, map(status))` — the worst-of, precedence `DECERR > SLVERR > OKAY`, with `status` encoding 0/1/2 = OK/SLVERR/DECERR (§??). MC Core supplies exactly one completion per accepted 64-B write packet (§??). `exp_pkts` is the count of 64-B MC packets the Burst Splitter generates for the transaction; given the 64-B-beat contract (§??) it equals `AWLEN + 1`.

D.2 Ordered response and release

The head index of each per-AXID Slot FIFO is eligible when `completion_bitmap` bits `[exp_pkts-1:0]` are all set. A priority selection loads an eligible index into the write Retiring Register. The write response logic presents one BRESP for that transaction and holds it until `bresp_ready`. Only on that acceptance does retirement pop the selected Slot FIFO and return `ROB_INDEX` to the write Free List.

This provides per-AXID AXI B ordering without coupling write release to read traffic.

E Read Completion Tracking and Serialization

Read completion tracking, final-beat indication, and release are implemented by the read ROB instance and its Retiring Register (§??).

E.1 Read ROB metadata and completion

The read Metadata RAM is directly indexed by `ROB_INDEX`. Each entry contains `exp_pkts[6:0]` (the number of 64-B MC packets for this transaction, 1..64), a 64-bit completion bitmap, and `rd_dbuf_base`, the base of a contiguously-allocated RD_SRAM range of `exp_pkts` 64-B lines. The count is initialized from the Burst Splitter — one MC packet per 64-B AXI data beat. The read entry stores no AXI address or 64 B line footprint: same-line read/write ordering is enforced entirely at read admission, before allocation (§??), and a completed read is never re-checked.

On a read completion, `GLOBAL_TAG[13:0]` directly selects the entry (`[13:6]`) and its packet-number bit (`[5:0]`). The returned 64-B data unit is written to `rd_dbuf_base + PACKET_NUMBER` in RD_SRAM. RD_SRAM depth (hence `RD_SRAM_ADDR.W`), its allocation/read/free handshakes, and latency remain **TBD**; the ROB assumes only that the contiguous `rd_dbuf_base` range is valid from allocation until the contract-defined release point.

E.2 Read retirement

An AXID Slot FIFO head can retire only after `completion_bitmap` bits `[exp_pkts-1:0]` are all set. The Retiring Register then holds that index while the Read Serializer emits `rd_dbuf_base + 0 .. rd_dbuf_base + exp_pkts-1` in packet order, one AXI beat per 64-B MC packet. `rob_last` is generated with the final beat (`PACKET_NUMBER == exp_pkts - 1`). Final AXI R acceptance, after the `rd_dbuf_base` range is released, permits retirement to pop the Slot FIFO and return `ROB_INDEX` to the read Free List. Completion alone never frees the index.

F Read and Write Reorder Buffers

F.1 Overview

The ROB is a generic transaction-tracking core, instantiated independently for the read path and the write path. Each instance owns a complete, separate copy of every structure described in §?? — its own 256-entry Free List, its own 256-entry Metadata RAM, and its own bank of 64 Per-AXID Slot FIFOs (depth 4 each). Nothing is shared between the two instances. They differ only in what a completed transaction produces: the read instance additionally holds a Data Buffer association and drives a Read Serializer; the write instance holds an aggregate response-status field and a 64 B line footprint consumed by read admission (§??), and drives BRESP generation directly, since a write completion carries no payload.

Each transaction is identified end-to-end by a single `GLOBAL_TAG`, described in §??. `ROB_INDEX` is the Metadata RAM address end to end.

A transaction becoming *complete* (all its packets accounted for in `completion_bitmap`) is not the same event as a transaction being *freed*. A completed transaction remains allocated while its ordered AXI response is being emitted and accepted, and (read only) while its contiguous RD_SRAM range is still being drained. §?? makes this lifecycle explicit.

Lifecycle Summary

The full life of a ROB entry, in order, referenced to the sections that detail each stage:

1. AXI request accepted (upstream of this file) — §??
2. `ROB_INDEX` allocated from the Free List — §??, §??

3. Metadata RAM initialized (`exp_pkts`, `completion_bitmap` = 0, (read) `rd_dbuf_base`) — §??
4. Entry becomes visible in its AXID’s Slot FIFO — §??, §??
5. Packets emitted, `GLOBAL_TAG` attached — §??
6. Completions decoded, `completion_bitmap` updated — §??, §??
7. Transaction reaches completion; becomes eligible to retire — §??
8. Ordered AXI response emitted and accepted (read: beat by beat; write: single BRESP) — §??, §??
9. Slot FIFO popped, `ROB_INDEX` freed, (read) `RD_SRAM` range already released — §??, §??

Architecture principles

- Completion directly supplies `ROB_INDEX` and `PACKET_NUMBER`; RAM access is a bit-slice decode.
- Per-AXID ordering is carried entirely by the Per-AXID Slot FIFOs (§??).
- Packet position is the low 6 bits of `GLOBAL_TAG` (`PACKET_NUMBER`).
- Read request-issue backpressure is a function of the pooled Read Data Buffer’s own 128-entry watermark, a separate structure outside this file’s scope (see §??). No watermark applies to Metadata RAM.

G Global Tag

Field	Width
<code>ROB_INDEX</code>	8 bits (addresses one of 256 Metadata RAM entries)
<code>PACKET_NUMBER</code>	6 bits (64-B MC packet index, 0..63)
<code>GLOBAL_TAG</code>	14 bits, { <code>ROB_INDEX</code> [7:0], <code>PACKET_NUMBER</code> [5:0]}

Generation. At allocation, the Allocator (§??) issues `ROB_INDEX`. At packet emission, the Burst Splitter supplies `PACKET_NUMBER` (0..`exp_pkts`-1) for each 64-B MC packet of the transaction. Packet Formation concatenates the two into `GLOBAL_TAG`, which is attached to every packet sent to MC Core.

Carry. MC Core is not required to interpret `GLOBAL_TAG` — it echoes the value back unmodified, *exactly one echo per accepted 64-B request packet*, as required by §?? (item 1). The `status`[3:0] to AXI RESP mapping is frozen in §??.

Decode. On completion, Completion Input (§??) splits the returned `GLOBAL_TAG` directly: bits [13:6] address Metadata RAM (`ROB_INDEX`); bits [5:0] select the packet position within that entry (`PACKET_NUMBER`). Both are pure bit-slices — no lookup, no associative match.

H Generic Architecture

The blocks in this section are instantiated once per ROB instance (i.e. once for read, once for write), with identical logic and identical sizing in both — see Overview.

H.1 Allocator / Free List

- 256-bit free/occupied bitmask, one bit per `ROB_INDEX`. 1 = free, 0 = occupied.
- Allocate: priority encoder selects lowest free bit, clears it, and outputs `ROB_INDEX`[7:0].

- Free: Retirement Logic sets the bit at `ROB_INDEX` via `free_req`, only once a slot has fully retired (§??) — never merely on completion.
- `FREE_LIST_EMPTY` (all bits 0) is a raw pool-wide allocation-backpressure signal. With `ROB_DEPTH` = 256 and $64 \times \text{depth-4 Slot FIFO capacity}$, it means every transaction slot is allocated and all FIFO capacity is collectively exhausted. A single full Slot FIFO does not imply this condition. See §??.

H.2 Metadata RAM

- 256-entry RAM, directly addressed by `ROB_INDEX` (no CAM, no translation).
- Depth is fixed at `ROB_DEPTH` = 256; no watermark or occupancy threshold applies to this RAM (see Overview). Allocation is gated by Free List exhaustion or per-AXID Slot FIFO fullness (§??); these are distinct conditions.
- Entry contents are defined explicitly in §??.
- Write port A (allocation): Allocator writes `exp_pkts`, clears `completion_bitmap`, (read instance) writes `rd_dbuf_base`, and (write instance) writes the `wr_line_base` and `wr_line_count` footprint (§??) at `ROB_INDEX` on allocation, before the entry is exposed to retirement — see §?? for the exact ordering.
- Write port B (completion): Completion Input sets one `completion_bitmap` bit (both instances) and, for the write instance, folds `status` into `resp_status`, at `ROB_INDEX`. `rd_dbuf_base` is not written on completion — it is consumed to place the returned 64-B data unit at `rd_dbuf_base` + `PACKET_NUMBER`.
- Read port: Retirement Logic reads the entry at the head-of-FIFO `ROB_INDEX` for each AXID to evaluate completion.
- No explicit `valid` bit. A Metadata RAM entry is only meaningful while its `ROB_INDEX` is occupied (cleared in the Free List) and present in a Per-AXID Slot FIFO; both conditions are tracked externally to the RAM.

H.3 Per-AXID Slot FIFO

- 64 independent FIFOs, one per AXID, depth 4 each — frozen sizing, identical for both instances. Each entry stores `ROB_INDEX[7:0]`.
- Push: only after the corresponding Metadata RAM entry has been initialized (`exp_pkts`, `completion_bitmap` = 0, (read) `rd_dbuf_base`) — see §??. An entry is never visible in a Slot FIFO before it is safe to read.
- Pop: not on completion. Only when the transaction at the head has fully retired — i.e. its ordered AXI response has been emitted and accepted (§??).
- FIFO order = allocation order = required AXI response order for that AXID, by construction. No separate ordering structure is needed.
- FIFO-full for a given AXID is the per-AXID backpressure condition on allocation. Because a completed-but-not-yet-retired entry remains at the head until fully drained, the head slot may stay occupied for several cycles after completion during response emission — this is expected, not a fault condition.

H.4 Completion Input

- Receives the MC completion/response unit together with `GLOBAL_TAG` and `status`. The exact response transport, payload width, and handshake are defined by the MC–CIF interface contract.

- Decodes `GLOBAL_TAG` into `ROB_INDEX` and `PACKET_NUMBER` per §??.
- Sets `completion_bitmap[PACKET_NUMBER] = 1` at `ROB_INDEX`.
- Read instance: writes the returned 64-B data unit to `rd_dbuf_base + PACKET_NUMBER` in `RD_SRAM` (the base was recorded at allocation, not derived here). The Data Buffer write protocol is defined by the MC/Data-Buffer interface contract — see §??.
- Write instance: folds `status` into `resp_status` (worst-of; precedence `DECERR > SLVERR > OKAY`, encoding frozen in §??) — see §??.
- Completion acceptance follows the MC-CIF response interface contract. The ROB does not impose a separate completion-side handshake beyond that contract.
- Completion Input only ever marks *completion*. It never pops a Slot FIFO and never frees a `ROB_INDEX` — both are the exclusive responsibility of Retirement Logic (§??).

H.5 Retirement Logic

Completion and retirement are distinct events. Completion (`completion_bitmap` full) makes a transaction *eligible* to retire; it does not by itself release anything. Release happens only after the transaction’s ordered AXI response has been fully accepted.

- **Retiring Register** (generic, minimal addition): a single register per instance holding the `ROB_INDEX` currently being drained, plus read-instance progress state consisting of a 6-bit `PACKET_NUMBER` packet pointer and any within-beat progress state required by the MC/Data-Buffer interface contract. The write instance requires no analogous data-stream progress state. At most one transaction is draining at a time per instance, consistent with the single response-channel path each instance already serves.
- Each cycle, while the Retiring Register is idle, Retirement Logic reads the Metadata RAM entry at the head `ROB_INDEX` of every AXID’s Slot FIFO and checks that `completion_bitmap` bits `[exp_pkts-1:0]` are all set (for `exp_pkts = 64`, the all-ones condition).
- When idle, a priority encoder selects the lowest-numbered AXID whose head entry is complete and loads its `ROB_INDEX` into the Retiring Register, entering the *retiring* state. **The Slot FIFO is not popped and `ROB_INDEX` is not freed at this point.**
- While retiring, instance-specific emission proceeds (§?? for read, §?? for write) until the transaction’s full AXI response has been accepted downstream (`merge_ready` for read, `bresp_ready` for write — §??).
- Only once the full response has been accepted: Retirement Logic pops that AXID’s Slot FIFO entry, returns `ROB_INDEX` to the Free List (`free_req`), and the Retiring Register returns to idle.
- Maximum one transaction draining at a time per instance; draining a single read transaction spans one AXI beat per 64-B MC packet, up to $\text{exp_pkts} \leq 64$ beats, unlike the previous single-cycle-emit assumption.

I Read-Instance Extension

State and logic in this section exist only in the read-path ROB instance.

I.1 Metadata RAM Extension

- `rd_dbuf_base[RD_SRAM_ADDR_W-1:0]`: base address of a single contiguously-allocated `RD_SRAM` range of `exp_pkts` 64-B lines. The returned data for packet `PACKET_NUMBER` occupies `rd_dbuf_base`

+ `PACKET_NUMBER`. Written once at allocation (§??); *not* written by Completion Input, which consumes it to place the response.

- `RD_SRAM_ADDR_W` is **TBD**: `RD_SRAM` depth is not specified in any current document (§??). The ROB treats `rd_dbuf_base` as an opaque base pointer whose only defined operation is + `PACKET_NUMBER`.

I.2 Read Data Buffer (cross-reference)

- The pooled Read Data Buffer (`RD_SRAM`), its independently managed 128-entry watermark, and its occupancy/backpressure logic are **out of scope for this file**. It is a separate structure the read ROB addresses through `rd_dbuf_base`, not a substructure of Metadata RAM. Data-buffer capacity and watermark are independent of `ROB_DEPTH`/Metadata RAM capacity.
- Exact Data Buffer interface signal names, allocation representation, and timing are **TBD** in the MC/Data-Buffer interface contract — see §??.

I.3 Read Serializer

- Operates only while the Retiring Register (§??) holds a read transaction.
- Serializes the completed data at `rd_dbuf_base` + `PACKET_NUMBER` for `PACKET_NUMBER` = 0 .. `exp_pkts-1` in packet order, tagged with the AXID selected from the Slot FIFO context. Each 64-B MC packet is one AXI data beat; the within-beat handshake to the merge path follows the MC/Data-Buffer interface contract.
- `merge_ready` accepts the AXI beat currently presented (one per 64-B MC packet). The contiguous `rd_dbuf_base` range is released as a single unit once the final beat has been accepted.
- `rob_last` is asserted on the final beat, i.e. when `PACKET_NUMBER == exp_pkts - 1` is accepted.
- Once the final beat is accepted and the `rd_dbuf_base` range is released, Retirement Logic pops the Slot FIFO and frees `ROB_INDEX` (§??).

Read-Instance I/O (in addition to generic I/O in §??)

Signal	Width	Dir / Peer	Description
<code>data</code>	512	in, MC Core	Read response data, one 64-B unit per completed 64-B MC packet
<code>data</code>	512	out, Merge Logic	In-order read response data unit (one 64-B AXI R-channel beat)
<code>status[3:0]</code>	4	out, Merge Logic	Status for emitted beat
<code>rob_last</code>	1	out, Merge Logic	Asserted on the final AXI beat (<code>PACKET_NUMBER == exp_pkts - 1</code>)
<code>merge_ready</code>	1	in, Merge Logic	Accept for the AXI beat currently emitted by the Read Serializer

Table 9: Read-instance additional signals

J Write-Instance Extension

State and logic in this section exist only in the write-path ROB instance.

J.1 Metadata RAM Extension

- **resp_status**[1:0]: aggregate **BRESP** value for the transaction. Initialized to **OKAY** at allocation; updated by Completion Input as each packet’s status arrives, worst-of semantics (priority ordering **DECERR** > **SLVERR** > **OKAY**, encoding frozen in §??).
- **wr_line_base**[25:0]: start 64 B line number (**ADDR_W** - 6) of the write’s footprint.
- **wr_line_count**[6:0]: contiguous 64 B line span of the write, in [1,64].

Both footprint fields are written by the allocation path from {**AWADDR**, **AWLEN**, **AWSIZE**, **AWBURST**} (§??), are never written by Completion Input, are read only by the read-admission same-line hazard check, and remain valid for the whole allocated lifetime of the entry — released with it (§??).

J.2 BRESP Generation

- Operates only while the Retiring Register (§??) holds a write transaction.
- Emits a single **bresp** = **resp_status** to the Response Dispatcher, tagged with the **AXID** currently being serviced, and holds it until accepted downstream (**bresp_ready**, §??).
- No data movement, no per-packet serialization — one response per transaction regardless of **exp_pkts**.
- Once **bresp_ready** accepts the response, Retirement Logic pops the Slot FIFO and frees **ROB_INDEX** (§??) — not before.

Write-Instance I/O (in addition to generic I/O in §??)

Signal	Width	Dir / Peer	Description
bresp [1:0]	2	out, Response Dispatcher	Aggregate write response for the transaction
bresp_ready	1	in, Response Dispatcher	Accept for bresp ; gates Slot FIFO pop and ROB_INDEX free in Retirement Logic

Table 10: Write-instance additional signals

K Generic I/O

Signal	Width	Dir / Peer	Description
alloc_req	1	in, Ac- cept/Request logic	Pulse: allocate a new ROB slot
AXID [5:0]	6	in, Ac- cept/Request logic	AXID of the transaction being allocated; used only to select the Per- AXID Slot FIFO, not stored in Metadata RAM
exp_pkts [6:0]	7	in, Burst Splitter	Number of 64-B MC packets for this transaction, 1..64
ROB_INDEX [7:0]	8	out, Ac- cept/Request logic	Allocated slot index, returned for use in GLOBAL_TAG generation

MC_RESP	TBD	in, MC Core	MC completion/response unit; exact transport defined by the MC-CIF interface contract
GLOBAL_TAG[13:0]	14	in, MC Core	{ROB_INDEX[7:0], PACKET_NUMBER[5:0]}, echoed unmodified
status[3:0]	4	in, MC Core	Per-packet completion status; 0/1/2 = OK/SLVERR/DECERR, codes 3..15 reserved → DECERR (§??)

Table 11: Generic signals, present on both instances

Instance-specific downstream accept signals (`merge_ready`, `bresp_ready`) are listed with their respective instance’s I/O table above, since only one instance’s Retirement Logic consumes each.

L Metadata RAM Entry Definition

Field	Width	Scope
<code>exp_pkts[6:0]</code>	7 bits	generic
<code>completion_bitmap[63:0]</code>	64 bits	generic
<code>rd_dbuf_base</code>	RD_SRAM_ADDR_W bits (TBD)	read-only
<code>resp_status[1:0]</code>	2 bits	write-only
<code>wr_line_base[25:0]</code>	26 bits	write-only
<code>wr_line_count[6:0]</code>	7 bits	write-only

Total entry width: 71 + RD_SRAM_ADDR_W bits (read instance — `exp_pkts` 7 + `completion_bitmap` 64 + `rd_dbuf_base`), 106 bits (write instance — 71 generic + 2-bit `resp_status` + the 33-bit 64 B line footprint, §??). AXID is available from Slot FIFO context at retirement and is not stored in the entry. The write entry stores the transaction’s 64 B line footprint; `rd_dbuf_base` is the read entry’s only address-like field and is an opaque RD_SRAM base. No ”retiring” state is stored per entry either — that state lives entirely in the single Retiring Register described in §??, since at most one transaction per instance is ever draining at a time.

M Allocation Path

1. Accept/Request logic sends `alloc_req` with AXID, following AXI request acceptance upstream of this file. On the read path, `alloc_req` is withheld while the AR Metadata FIFO head is held by the same-line hazard check (§??); no read entry is allocated during the hold.
2. Allocator pops ROB_INDEX from the Free List (priority encoder on the bitmask), clears the bit. ROB_INDEX is now reserved but not yet visible to Retirement Logic.
3. Once Burst Splitter Stage 1 computes `exp_pkts` (the number of 64-B MC packets, 1..64) for the transaction, Metadata RAM is written at ROB_INDEX: `exp_pkts` set, `completion_bitmap` cleared to 0, (read instance) `rd_dbuf_base` set to the base of a contiguous `exp_pkts`-line RD_SRAM allocation, and (write instance) `resp_status` initialized to OKAY with `wr_line_base` and `wr_line_count` written from {AWADDR, AWLEN, AWSIZE, AWBURST} (§??).

4. Only *after* this Metadata RAM initialization write completes is `ROB_INDEX` pushed onto the requesting AXID's Slot FIFO. This ordering is deliberate: the Slot FIFO push and the Metadata RAM initialization write are both gated on the same Burst Splitter Stage 1 completion, so an entry can never become visible to Retirement Logic with a stale or uninitialized `exp_pkts/completion.bitmap`.

N Packet Issue Path

1. For each 64-B MC packet `PACKET_NUMBER = 0..exp_pkts-1` emitted by the Burst Splitter:
2. Packet Formation attaches `GLOBAL_TAG = {ROB_INDEX[7:0], PACKET_NUMBER[5:0]}` to the packet. No per-packet Read Data Buffer handle is recorded — the contiguous `rd_dbuf_base` range was allocated at §??, and packet `PACKET_NUMBER`'s data will land at `rd_dbuf_base + PACKET_NUMBER`.
3. Packet is issued to MC Core via the Packet FIFO.

O Completion Path

1. The MC response unit arrives with `GLOBAL_TAG`, `status`, and, for reads, response data according to the MC-CIF interface contract.
2. Completion Input decodes `GLOBAL_TAG` into `ROB_INDEX` and `PACKET_NUMBER`.
3. (read instance) the returned 64-B data unit is written to `rd_dbuf_base + PACKET_NUMBER` in RD-SRAM.
4. (write instance) `resp_status` is updated with the worst-of comparison against the incoming `status`.
5. `completion_bitmap[PACKET_NUMBER]` is set at `ROB_INDEX`.
6. Completion acceptance only marks the corresponding packet complete; it never triggers a Slot FIFO pop or a Free List return. Retirement remains responsible for releasing the ROB entry.

P Retirement Path

1. Each cycle, while the Retiring Register is idle, Retirement Logic reads the Metadata RAM entry at every AXID's Slot FIFO head.
2. Priority encoder selects the lowest AXID whose head entry has `completion.bitmap` bits `[exp_pkts-1:0]` all set; its `ROB_INDEX` is loaded into the Retiring Register. **The Slot FIFO entry is not popped yet.**
3. (read instance) Read Serializer drains `rd_dbuf_base + 0 .. rd_dbuf_base + exp_pkts-1` in packet order, one AXI beat per 64-B MC packet.
4. (write instance) BRESP Generation emits `bresp` and holds it until `bresp_ready` accepts it.
5. Only once the transaction's full AXI response has been accepted (final read beat, or the single write response) does Retirement Logic pop that AXID's Slot FIFO and return `ROB_INDEX` to the Free List. The Retiring Register returns to idle.

Q Slot Release

1. Triggered only by full AXI response acceptance (§??) — never by `completion.bitmap` reaching its full mask on its own.

2. ROB_INDEX is returned to the Free List (**free_req**), bit set back to 1 = **free**.
3. (write instance) the **free_req** pulse re-arms the same-line hazard recheck for any AR Metadata FIFO head currently held (§??); the freed entry's **wr_line_base** and **wr_line_count** cease to be occupied in the same cycle.
4. (read instance) the contiguous **rd_dbuf_base** range (**exp_pkts** lines) is returned to RD_SRAM as a single unit, once the final AXI beat has been accepted.
5. Retirement needs only the Slot FIFO pop and Free List return.

R Backpressure

Signal	Description
Slot FIFO full (per AXID)	Blocks new allocation for that AXID. A completed-but-still-draining head entry keeps the FIFO occupied until fully retired, so this condition can now persist slightly longer than "completion" alone would suggest — expected behavior, not a fault.
FREE_LIST_EMPTY	Blocks new allocation pool-wide. It means all 256 transaction slots are allocated and all Slot FIFO capacity is collectively consumed. A single full AXID FIFO does not imply Free List exhaustion.
AR Metadata FIFO head held	Read path only. The head read overlaps an occupied ROB_WR 64 B line footprint (§??); read admission on that port is held until every overlapping write has retired. No write-path equivalent.

ROB_DEPTH = 256 carries no watermark of its own. Read-path issuance backpressure is driven entirely by the pooled Read Data Buffer's 128-entry watermark, a separate structure not detailed in this file.

S Key Parameters

Parameter	Value
Metadata RAM depth (ROB_DEPTH)	256 entries (ROB_INDEX is 8 bits) — frozen, identical for both instances
PACKET_NUMBER width	6 bits (MAX_PKTS = 64) — frozen
GLOBAL_TAG width	14 bits — frozen
exp_pkts width	7 bits (packet count 1..64) — frozen
completion_bitmap width	64 bits — frozen
Entry width, read instance	71 + RD_SRAM_ADDR_W bits (RD_SRAM_ADDR_W TBD)
Entry width, write instance	106 bits (71 generic + 2-bit resp_status + 33-bit 64 B line footprint, §??)
Free list width	256 bits, one independent Free List per instance
Per-AXID Slot FIFO	64 FIFOs × depth 4 — frozen, identical for both instances
Retiring Register	1 per instance: ROB_INDEX[7:0] + 6-bit read packet pointer + within-beat
Max transactions draining	1 per instance at a time; a single read transaction's drain spans one AXI b

$64 \times 4 = 256$, exactly matching `ROB_DEPTH`. Thus `FREE_LIST_EMPTY` means all per-AXID FIFO capacities are collectively occupied; it remains distinct from the condition that any one FIFO is full.

T Unresolved / TBD

Every item below requires an external decision or a separate component’s specification — none can be derived from the frozen ROB architecture itself.

- **MC error-code assignment (codes 3..15).** The interface `status[3:0]` field and its AXI mapping are frozen (§??): 0/1/2 = OK/SLVERR/DECERR, and any received value in 3..15 is treated by CIF as DECERR. Which specific fault classes MC Core assigns to codes 3..15, if any, is an MC-owned detail and does not affect CIF RTL.
- **Pooled Read Data Buffer / MC response contract.** Signal names, allocation representation, and timing for allocating, filling, serializing, and freeing the read data range are defined by a separate MC/Data-Buffer interface specification. This file assumes only that the contiguous `rd_dbuf_base` range is valid from allocation until the contract-defined release point, and that packet N’s returned 64-B data unit occupies `rd_dbuf_base + N`.
- **RD_SRAM depth / RD_SRAM_ADDR.W.** The depth of RD_SRAM, and therefore the width of `rd_dbuf_base` and the read-instance entry, is not specified in any current document. The ROB carries `rd_dbuf_base` as an opaque base pointer; RD_SRAM’s depth, occupancy model, and watermark are out of scope for this file. The RD_SRAM completion-side write port must be available whenever a RD_DATA response is presented (§??, response-side flow control).
- **MC write-completion safety (interface requirement).** The property that a write reported complete is visible to a subsequent same-line read is stated as an interface requirement on MC Core (§??, item 2), *not* asserted to be implemented. The current MC-core documentation additionally contains an unresolved contradiction about how long a write remains hazard-visible (its hazard-unit admission-station dwell versus its full buffer-to-DRAM lifetime); the MC team must resolve that before the requirement can be signed off. Until then, the conservative interpretation — `WR_ACK` at DRAM commit — satisfies the requirement without any MC hazard-unit dependency.

U Timing Notes

- Metadata RAM read/write ports: allocation write, completion write, and retirement read are independent accesses; simultaneous allocation and completion in the same cycle to different `ROB_INDEX` values is safe, unchanged in spirit from the prior BRAM port arrangement.
- Priority encoder across 64 Slot FIFO heads in Retirement Logic remains the likely critical path; may need pipelining at high frequencies, as in the prior design. It only runs while the Retiring Register is idle.
- Simultaneous `alloc_req` and `free_req` on the Free List bitmask in the same cycle is safe — they target different bits by construction (allocate clears the lowest free bit; free sets the bit of the just-retired, and therefore currently occupied, slot).
- `GLOBAL_TAG` decode (§??) is a pure bit-slice and has no translation latency on the completion path.
- The Slot FIFO push at allocation is deliberately deferred until the Metadata RAM initialization write completes (§??) — this introduces a small, bounded latency between `alloc_req` and Slot FIFO visibility, bounded by Burst Splitter Stage 1’s latency to produce `exp_pkts`.

- Draining a read transaction is explicitly multi-cycle through the Retiring Register. Transfer granularity is one AXI beat per 64-B MC packet, so Retirement Logic and the Read Serializer must be read as a small state machine (idle / retiring), not a single combinational selection. The write instance is analogous but single-beat, gated by **bresp_ready**.